

```
1 import 'package:flutter/material.dart';
2 import 'package:hacker_news_api/components/info_snack_bar.dart';
3 import 'package:hacker_news_api/pages/partials/custom_app_bar.dart';
4 import 'package:hacker_news_api/pages/partials/error_info_page.dart';
5 import 'package:hacker_news_api/pages/partials/loading_page.dart';
6 // import story_model.dart
7 import 'package:hacker_news_api/models/story_model.dart';
8 import 'package:hacker_news_api/services/hacker_news_service.dart';
9
10 /// classe HomePage
11 /// Widget di alto livello che coordina lo stato,
12 /// la logica di caricamento dati e la composizione
13 /// dei componenti grafici della HomePage
14 class HomePage extends StatefulWidget {
15   /// costruttore HomePage
16   const HomePage({super.key});
17
18   /// override del metodo createState
19   @override
20   State<HomePage> createState() => _HomePageState();
21 }
22
23 /// classe HomePageState
24 class _HomePageState extends State<HomePage> {
25   /// variabile di stato storyModelFuture che conterrà
26   /// un qualcosa che verrà risolto in futuro e quel
27   /// qualcosa è un'istanza di StoryModel
28   late Future<List<StoryModel>> storyModelFuture;
29
30   /// "service" variabile privata che contiene
31   /// un'istanza di "HackerNewsService"
32   /// e sarà utilizzata per invocare
33   /// i metodi di quella classe
34   final _service = HackerNewsService();
35
36   /// Cache dell'ultima lista valida mostrata.
37   /// Se non è null, NON mostriamo mai LoadingPage durante refresh.
38   List<StoryModel>? _storiesCache;
39
40   /// variabile di stato che rappresenta
41   /// se c'è un refresh in corso o meno.
42   /// Inizializzata a "false", perché all'avvio
43   /// non c'è nessun refresh in corso.
44   /// Stato iniziale: non sto ricaricando nulla
45   bool _refreshInProgress = false;
46
47   /// override del metodo initState
48   @override
49   void initState() {
50     // super initState
51     super.initState();
52
53     /// Inizializzare i dati della pagina.
54     /// Serve sapere quando finisce il Future
55     /// per popolare la cache, senza mostrare SnackBar
56     _loadInitial();
57   }
58
59   /// Metodo che serve a caricare i dati al primo avvio
60   /// della pagina, mostrando il "LoadingPage", senza
```

```
61  /// SnackBar, e popolando la cache quando i dati arrivano.
62  /// Diverso dal refresh manuale perché:
63  /// - non deve mostrare feedback all'utente (SnackBar)
64  /// - deve solo "portare la pagina da vuota -> lista".
65  /// Qui non decido cosa mostrare ma preparo lo stato.
66  /// La UI reagisce leggendo quello stato.
67  /// Scopo: inizializzare i dati della pagina.
68  /// Avvia il caricamento iniziale, mostra il loading,
69  /// quando i dati arrivano aggiorna lo stato,
70  /// se fallisce lascia che la UI gestisca l'errore
71  void _loadInitial() {
72    /// Avvio del download
73    /// Avvio chiamata asincrona + salvo il Future nello stato
74    /// -> il FutureBuilder reagisce.
75    /// Avvio del Future
76    storyModelFuture = _service.downloadStoryData();
77
78    storyModelFuture
79      /// quando il Future va a buon fine,
80      /// aggiorno la cache e faccio rebuild.
81      /// ".then" = quando questo Future
82      /// si completa con successo, esegui questa funzione.
83      /// "stories" = risultato finale "List<StoryModel>".
84      /// Quando il Future termina...
85      .then((stories) {
86        /// se questa pagina non esiste più, non fare nulla
87        if (!mounted) return;
88
89        /// aggiornamento dello stato (cache) +
90        /// Flutter fa un rebuild della UI.
91        /// Aggiorna lo stato dell'applicazione...
92        setState(() {
93          _storiesCache = stories;
94        });
95      })
96      /// quando il Future fallisce,
97      /// non faccio nulla -> la UI gestisce l'errore.
98      /// Intercetta solo errori del Future, evita crash e
99      /// non mostra SnackBar
100     .catchError((Object e, StackTrace s) {
101       /// Qui non serve SnackBar:
102       /// il builder mostrerà ErrorInfoPage al primo avvio
103
104       /// log del messaggio di errore
105       debugPrint("Errore in _loadInitial: $e");
106
107       /// log dello stack trace
108       debugPrint(s.toString());
109     });
110  }
111
112  /// Metodo che serve a gestire un refresh manuale
113  /// (tap sul bottone in AppBar o pull-to-refresh)
114  /// con questi obiettivi:
115  /// - evitare refresh multipli in parallelo (spam di tap/gesture)
116  /// - far partire una nuova richiesta al service
117  /// e aggiornare "storyModelFuture"
118  /// (così il "FutureBuilder" sa che c'è un nuovo caricamento)
119  /// - Aspettare il risultato della richiesta
120  /// (così il "RefreshIndicator" resta attivo
```

```
121 /// finché il download finisce)
122 /// - Se "successo": aggiornare la cache ("_storiesCache")
123 /// e mostrare relativa SnackBar
124 /// - Se "errore": mantenere la lista precedente (cache)
125 /// e mostrare relativa SnackBar
126 /// - Alla fine: sbloccare il refresh ("_refreshInProgress = false")
127 /// sempre, in ogni caso
128 Future<void> refreshData() async {
129   /// Se un refresh è già in corso,
130   /// ignora nuovi refresh.
131   /// Evita richieste multiple in parallelo, SnackBar multiple
132   /// e race condition
133   if (_refreshInProgress) return;
134
135   /// altrimenti, se non è in corso,
136   /// ne avii uno.
137   /// Accetto un refresh solo se non
138   /// ce n'è già uno attivo.
139   /// Segno che "da adesso in poi" un refresh è attivo.
140   /// Qui non serve "setState", perché questa variabile
141   /// non cambia la UI (serve solo come lock logico)
142   _refreshInProgress = true;
143
144   /// future specifico di questo refresh.
145   /// final -> lo assegni una sola volta
146   /// late -> lo assegni dopo (dentro "setState")
147   late final Future<List<StoryModel>> future;
148
149   /// Questo blocco fa due cose sincronizzate:
150   /// - avvia la richiesta
151   /// - aggiorna la variabile osservata dal "FutureBuilder".
152   /// Questo setState: è iniziato un refresh ->
153   /// aggiorno "storyModelFuture" -> "FutureBuilder" lo sa.
154   /// Notifico/avviso il "FutureBuilder" che è partito
155   /// un nuovo caricamento (stato di waiting, ...),
156   /// anche se poi non mostro il loading grazie alla cache
157   setState(() {
158     // avvio download/avvio richiesta
159     future = _service.downloadStoryData();
160     // notifica la UI
161     storyModelFuture = future;
162   });
163
164   try {
165     /// "await future" fa sì che:
166     /// - il metodo non termina subito
167     /// - lo spinner del "RefreshIndicator" resta visibile
168     /// finché finisce il download
169     /// - evito che la funzione torni prima del tempo
170     final freshStories = await future;
171
172     /// se la pagina è stata chiusa nel frattempo,
173     /// non devo toccare lo stato né mostrare SnackBar
174     if (!mounted) return;
175
176     /// Qui aggiorno la "fonte di verità" della lista visibile.
177     /// Flutter rifà build -> lista aggiornata.
178     /// Questo setState: "Sono arrivati i dati" ->
179     /// aggiorno "_storiesCache" -> ListView cambia
180     setState(() {
```

```
181     _storiesCache = freshStories;
182   });
183
184   /// Feedback all'utente: refresh completato
185   InfoSnackBar.show(
186     context,
187     message: "Lista aggiornata correttamente",
188     type: InfoType.success,
189     duration: const Duration(seconds: 2),
190   );
191 } catch (_) {
192   /// Se la richiesta fallisce, entri qui
193
194   /// Se la pagina non esiste più, non fare nulla
195   if (!mounted) return;
196
197   /// Mostrare messaggio di errore ma non si tocca la cache.
198   /// Risultato UX: l'utente resta con la lista vecchia
199   /// (se c'era) e sa che il refresh non è andato a buon fine
200   InfoSnackBar.show(
201     context,
202     message: "Errore durante l'aggiornamento",
203     type: InfoType.error,
204     duration: const Duration(seconds: 2),
205   );
206 } finally {
207   /// Sbloccare sempre (fondamentale)
208   _refreshInProgress = false;
209 }
210 }
211
212 /// override del metodo build
213 @override
214 Widget build(BuildContext context) {
215   /// Scaffold
216   return Scaffold(
217     /// appBar custom
218     appBar: CustomAppBar(onRefresh: refreshData),
219
220     /// body
221     body: Center(
222       /// child
223       child: body(),
224     ),
225
226     // colore di sfondo (Scaffold)
227     backgroundColor: Colors.white,
228   );
229 }
230
231 /// body: componente personalizzato
232 Widget body() => FutureBuilder<List<StoryModel>>(
233   /// future su cui ci si mette in ascolto
234   future: storyModelFuture,
235   // builder
236   builder: (context, snapshot) {
237     /// 1. Loading
238     /// SOLO primo avvio: nessuna cache + waiting
239     if (_storiesCache == null &&
240       snapshot.connectionState == ConnectionState.waiting) {
```

```
241 // ritorna la pagina di caricamento/loading
242 return const LoadingPage();
243 }
244
245 /// 2. Error
246 /// Se c'è errore al primo avvio (nessuna cache),
247 /// mostra pagina di errore
248 if (snapshot.hasError && _storiesCache == null) {
249   return ErrorInfoPage(
250     infoText: "Errore nel caricamento",
251     buttonText: "Riprova",
252     onRefresh: refreshData,
253   );
254 }
255
256 /// Da qui in poi: ho cache
257 /// (oppure sono in una situazione "strana")
258 final stories = _storiesCache;
259
260 /// 3. No data (null)
261 /// per sicurezza, in pratica non si dovrebbe quasi mai
262 /// arrivare qui
263 if (stories == null) {
264   return ErrorInfoPage(
265     infoText: "Nessun dato disponibile",
266     buttonText: "Ricarica",
267     onRefresh: refreshData,
268   );
269 }
270
271 /// 4. Empty list
272 if (stories.isEmpty) {
273   return ErrorInfoPage(
274     infoText: "Nessuna news trovata",
275     buttonText: "Aggiorna",
276     onRefresh: refreshData,
277   );
278 }
279
280 /// 5. Success (tutto ok)
281 /// Lista sempre visibile + pull-to-refresh
282 /// - Tap AppBar: nessuna LoadingPage, lista resta lì,
283 /// poi SnackBar
284 /// - Pull-to-refresh: spinner RefreshIndicator,
285 /// poi SnackBar
286 return RefreshIndicator(
287   /// Regola fondamentale del RefreshIndicator:
288   /// - "onRefresh" DEVE restituire un Future<void>
289   /// - lo spinner resta visibile finché il Future non termina
290
291   /// Collegamento tra il gesto dell'utente
292   /// (pull-down) e il metodo refreshData()
293   onRefresh: refreshData,
294
295   /// colore della freccia/spinner
296   color: Colors.white,
297
298   /// colore di sfondo del cerchio
299   backgroundColor: Colors.green,
300
```

```
301    /// child: ListView (variante separated)
302    child: ListView.separated(
303      /// la lista deve essere scrollabile anche se
304      /// il contenuto non riempie lo schermo
305      physics: const AlwaysScrollableScrollPhysics(),
306
307      /// itemCount
308      itemCount: stories.length,
309
310      /// itemBuilder
311      itemBuilder: (context, index) => ListTile(
312        /// title (titolo)
313        title: Text(
314          stories[index].title,
315          style: const TextStyle(fontWeight: FontWeight.bold),
316        ),
317
318        /// subtitle (autore)
319        subtitle: Text(
320          /// autore
321          stories[index].author,
322          /// stile di Text per autore
323          style: const TextStyle(fontStyle: FontStyle.italic),
324        ),
325      ),
326
327      /// separatorBuilder
328      /// divisore tra una news e l'altra
329      separatorBuilder: (context, index) =>
330        Divider(color: Colors.grey.shade300),
331    ),
332  );
333 },
334 );
335 }
336
```